

KeySiphon: Automated Side-Channel Key Recovery via Power Rail Eavesdropping

Umer Irfan, Abdullah Bin Faisal, Izza Naseer, Fahd Ibrahim Bhatti
 Syed Babar Ali School of Sciences and Engineering
 LUMS

Abstract—Cryptographic algorithms such as RSA are mathematically secure, yet their physical implementations on embedded devices may leak sensitive information through side channels. This project demonstrates a Simple Power Analysis (SPA) attack on a *AVR Series* microcontroller by exploiting shared power rail infrastructure; a realistic threat in IoT ecosystems where multiple devices coexist on common supply lines. We emulate a malicious co-resident device by isolating the target microcontroller’s VCC pin and measuring voltage fluctuations through a tuned $47\ \Omega$ shunt resistor network. This setup bypasses onboard decoupling capacitors and captures raw power consumption signatures corresponding to the Square-and-Multiply modular exponentiation algorithm. To automate key recovery at scale, we developed KeySiphon, a 1D convolutional neural network achieving 93.10% validation accuracy in distinguishing cryptographic operations from noisy power traces. This work demonstrates that even mathematically secure algorithms are vulnerable when co-located malicious hardware can monitor shared power infrastructure, and presents an end-to-end automated pipeline from signal acquisition to secret key extraction.

Index Terms—Side-Channel Analysis, Arduino Uno, Pin Lifting, Parallel Resistor Network, RSA, Hardware Security, Deep Learning

I. INTRODUCTION

Cryptographic primitives are the backbone of modern digital security. While algorithms like RSA are mathematically secure, physical devices executing them consume power in data-dependent ways, opening them up to Side-Channel Attacks (SCA) that infer secret keys through power fluctuations.

In modern IoT deployments, multiple devices often share common power infrastructure. This creates a realistic attack vector where a malicious co-resident device on the same power rail can passively monitor voltage fluctuations to extract cryptographic secrets. Unlike traditional SCA requiring direct chip access, this threat model exploits the shared nature of embedded system power distribution.

This project demonstrates Simple Power Analysis (SPA) on a *ATmega328P* by emulating a malicious monitor on a shared power rail. To isolate the target’s power consumption from onboard filtering capacitors, we physically lifted the VCC pin and inserted a shunt resistor network. We then developed KeySiphon, an automated deep learning pipeline that distinguishes between Square and Multiply operations in RSA modular exponentiation, achieving 93.10% key recovery accuracy.

The key contributions of this work are:

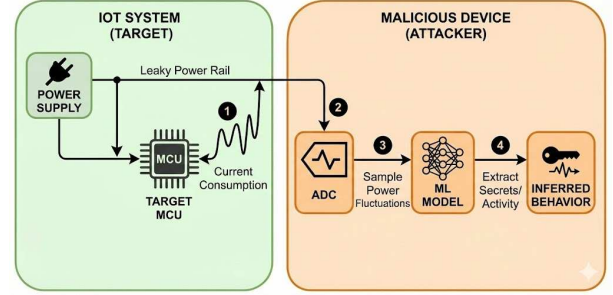


Fig. 1. KeySiphon system architecture showing the malicious monitoring device passively observing power fluctuations from the target MCU on a shared power rail.

- C1. A realistic threat model demonstrating that co-resident malicious hardware on shared IoT power infrastructure can extract cryptographic secrets through passive power rail monitoring, without requiring direct chip-level access.
- C2. KeySiphon, an automated deep learning pipeline achieving 93.10% accuracy in RSA key recovery from noisy power traces, scaling side-channel cryptanalysis beyond manual inspection to enable practical attacks on full-length cryptographic keys.

II. BACKGROUND

A. RSA and Square-and-Multiply

RSA encryption relies on modular exponentiation ($c = m^e \pmod{N}$), commonly implemented using the binary Square-and-Multiply algorithm where bit 0 performs only squaring and bit 1 performs squaring followed by multiplication. Since the CPU consumes distinct power levels for different operations, monitoring power consumption allows attackers to distinguish between these patterns and reconstruct the secret exponent bit-by-bit [1]. Messerges et al. demonstrated three attack classes (SEMD, MESD, ZEMD), with ZEMD requiring only algorithm and modulus knowledge [1]. Similar vulnerabilities exist in elliptic curve implementations [7].

B. Power Analysis Attacks

CMOS logic consumes power during state transitions according to $P_{dyn} = \alpha \cdot C \cdot V_{DD}^2 \cdot f$, where α represents data-correlated switching activity [2]. Placing a resistor in series with power supply converts current variations to voltage fluctuations ($V_{drop} = I(t) \cdot R_{net}$), with the goal of maximizing V_{drop} while preventing brownout [4].

Simple Power Analysis (SPA) directly interprets power traces to identify instruction sequences using single or few measurements [2], exploiting visible power differences between operations [5]. Differential Power Analysis (DPA) uses statistical methods across many traces, while Correlation Power Analysis (CPA) models relationships between power consumption and Hamming weight or distance [6]. These techniques have been successfully applied to various platforms including AES implementations [9], FPGAs [10], ARM Cortex-M microcontrollers [11], and timing attacks on RSA [8].

Deep learning, particularly CNNs, has emerged as a powerful automation tool for side-channel attacks, extracting features without manual engineering [12] and achieving higher success rates than traditional methods [13], [14]. In shared infrastructure scenarios, malicious co-resident devices can monitor power rails or electromagnetic emissions to extract secrets [3], [6], with similar vulnerabilities demonstrated in cloud environments [15] and multi-core processors [16].

III. METHODOLOGY

A. Target Platform and Threat Model

The target is an ATmega328P microcontroller on an Arduino Uno R3 executing RSA modular exponentiation. We emulate a malicious co-resident device by isolating the target's power consumption and measuring current through a shunt resistor, simulating an attacker monitoring a shared power rail in an IoT deployment.

B. Hardware Isolation via Pin Lifting

Pin 7 (VCC) and Pin 20 (AVCC) of the ATmega328P were physically bent outward and lifted, preventing socket contact. Lead wires were soldered directly to the lifted pins (Fig. 2), isolating the chip's power input from the board's 5V rail and its decoupling capacitors, which would otherwise smooth out high-frequency signals necessary for analysis.

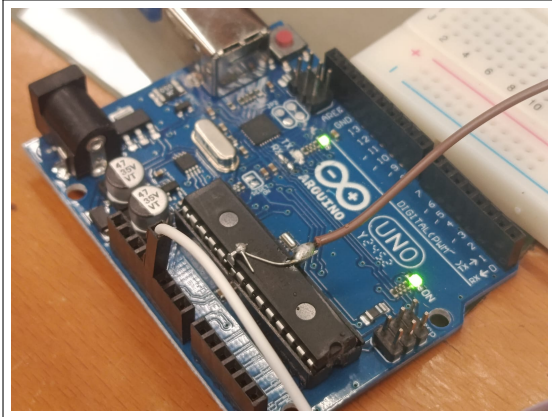


Fig. 2. Lifted VCC pin (Pin 7) with direct wire solder, isolating the chip from the PCB power plane.

C. Shunt Resistor Network and Brownout Prevention

The ATmega328P includes Brownout Detection (BOD) that resets the processor when supply voltage drops below approximately 4.3V. Inserting a shunt resistor creates a voltage drop $V_{drop} = I(t) \cdot R_{shunt}$ that must satisfy:

$$V_{MCU} = V_{supply} - I_{peak} \cdot R_{shunt} > V_{BOD} \quad (1)$$

With peak currents of 20-30mA, this theoretically limits resistance to $R_{max} \approx 23\Omega$. However, this creates a trade-off: higher resistance produces larger, more distinguishable signals but risks triggering BOD.

Through incremental testing with a parallel resistor network (Fig. 3), we empirically settled on 47Ω , which provides sufficient signal amplitude (50-150mV peaks) while maintaining adequate safety margin above the BOD threshold through conservative current estimates and the MCU's tolerance for brief voltage excursions.

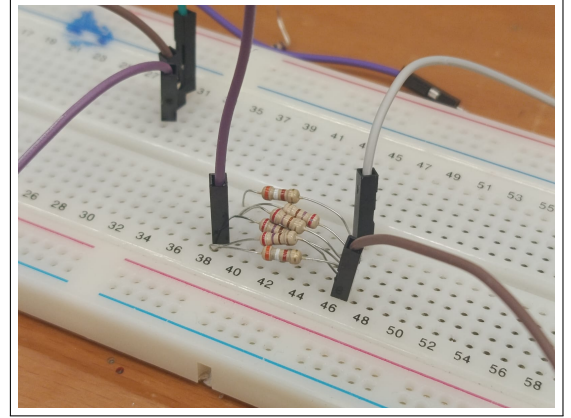


Fig. 3. Parallel resistor network providing 47Ω equivalent resistance.

D. Measurement Setup

An OWON oscilloscope measured voltage drop across the shunt resistor (high side: 5V supply; low side: VCC pin). A GPIO trigger synchronized captures with cryptographic operations. We captured a 5-minute powered-off baseline trace (Fig. 4) revealing 50Hz mains interference as the dominant noise source.

E. Software Implementation

The 16MHz clock executes operations faster than entry-level oscilloscopes can distinguish. To enable visual analysis, we injected artificial delays between operations (Listing 1). The `sleep_stage_timer` function introduces $500\mu s$ delays, creating distinct temporal plateaus for Squaring and Multiplication phases while preserving fundamental power consumption characteristics.

F. Proof-of-Concept vs. Deployment Scenario

Our experimental setup uses invasive hardware modification (pin lifting and shunt resistor insertion) to amplify power signatures for proof-of-concept validation with entry-level

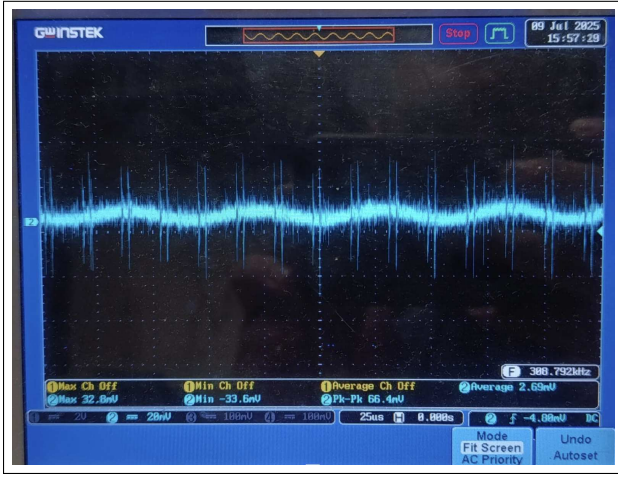


Fig. 4. Baseline noise floor showing 50Hz mains interference.

```

1 uint8_t modexp_sleep_8(uint8_t base, uint8_t exp, uint8_t
  mod) {
2   uint8_t result = 1;
3   base %= mod;
4
5   for (int bit = 7; bit >= 0; bit--) {
6     // Always SQUARE
7     result = modmul_8(result, result, mod);
8     sleep_stage_timer(delay_bw_stage);
9
10    // Conditional MULTIPLY (only if bit = 1)
11    if ((exp >> bit) & 1) {
12      result = modmul_8(result, base, mod);
13      sleep_stage_timer(delay_bw_stage);
14    }
15  }
16  return result;
17 }

```

Listing 1. Modified modular exponentiation with artificial delays

measurement equipment. In a realistic deployment, a malicious co-resident device would monitor voltage fluctuations directly on the shared power rail using high-resolution ADCs (16-bit or higher) and low-noise amplifiers. While the measured voltage variations would be smaller (microvolts rather than millivolts), the fundamental power consumption patterns remain identical. Our approach demonstrates a conservative lower bound on attack feasibility: if distinguishable with amplified signals, the attack remains viable with more sophisticated sensing hardware in non-invasive scenarios.

IV. RESULTS

A. Signal Acquisition

The pin-lifting technique successfully isolated the microcontroller’s power consumption from board-level noise filtering. The oscilloscope captured clean voltage fluctuations across the shunt resistor, with observable variations corresponding to cryptographic operations (Fig. 5).

B. Operation Pattern Recognition

Visual inspection of the power traces reveals distinct morphological signatures that directly correlate with the underlying algorithm structure:

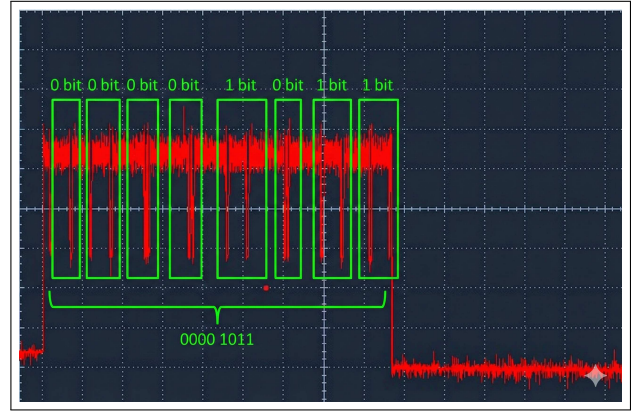


Fig. 5. Oscilloscope trace showing distinct voltage drops corresponding to cryptographic operations during modular exponentiation.

- **Single narrow bursts:** Correspond to isolated Squaring operations, indicating an exponent bit value of 0. These exhibit shorter duration and lower integrated energy consumption.
- **Double-width bursts:** Correspond to the Square-then-Multiply sequence, indicating an exponent bit value of 1. The extended duration and higher amplitude reflect the additional computational workload.

The $47\ \Omega$ shunt resistor configuration maintained voltage stability above the brownout threshold throughout all measurements, preventing system resets while providing sufficient signal amplitude for clear visual discrimination between operation types. The baseline idle state remained stable at approximately 4.85V, with active operation peaks dropping to 4.70-4.75V.

V. AUTOMATED ATTACK VIA DEEP LEARNING

While visual inspection suffices for demonstrating proof-of-concept attacks, manual analysis is impractical for production RSA keys of 1024 or 2048 bits. To enable scalable key recovery, we developed KeySiphon, a deep learning pipeline that automates the classification of power trace segments into their corresponding exponent bit values.

A. Data Preprocessing and Segmentation

Raw oscilloscope captures were segmented into fixed-length windows, each corresponding to one bit position in the modular exponentiation algorithm. This segmentation strategy isolates local temporal features, specifically the pulse width and amplitude differences between isolated Square operations (bit 0) and Square-then-Multiply sequences (bit 1) while filtering out long-term drift and environmental noise that would confound end-to-end trace analysis.

B. Neural Network Architecture

KeySiphon employs a 1D Convolutional Neural Network (CNN) designed for time-series classification (Fig. 6). The architecture consists of two primary components:

1) *Feature Extraction Pipeline*: Three stacked convolutional blocks progressively extract hierarchical temporal features:

- **Conv1d layers**: Apply 1D convolutions with kernel sizes [5, 5, 3] and filter counts [16, 32, 64], capturing multi-scale temporal patterns
- **BatchNorm1d**: Normalizes layer outputs to stabilize gradient flow during training
- **ReLU activation**: Introduces non-linearity for complex feature learning
- **MaxPool1d (stride=2)**: Downsamples feature maps while preserving dominant signal characteristics

2) *Classification Head*: The convolutional output is flattened into a 3,904-dimensional feature vector, then passed through:

- **Dropout (p=0.5)**: Randomly deactivates neurons during training to prevent overfitting
- **Fully connected layers**: Two dense layers map features to binary classification (bit 0 vs. bit 1)
- **Softmax output**: Produces class probabilities for final bit prediction

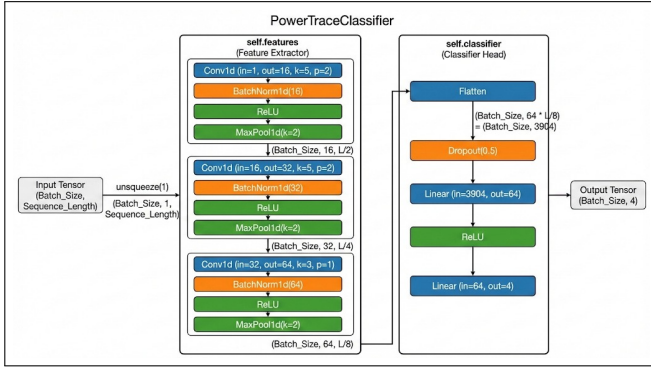


Fig. 6. KeySiphon CNN architecture showing the feature extraction pipeline (convolutional blocks) and classification head (fully connected layers).

C. Training Results and Performance Analysis

The model was trained for 20 epochs using the Adam optimizer with binary cross-entropy loss. The dataset was split 80/20 into training and validation sets, with each set containing balanced examples of both bit classes to prevent bias.

Figure 7 demonstrates rapid convergence, with both training and validation losses stabilizing after approximately 15 epochs. Table I summarizes final performance metrics.

TABLE I
KEYSIPHON MODEL PERFORMANCE AFTER 20 EPOCHS

Metric	Training	Validation
Loss	0.0353	0.0547
Accuracy	95.78%	93.10%

The 2.68% gap between training and validation accuracy indicates slight overfitting, likely due to the model memorizing noise patterns specific to the training traces. However, the

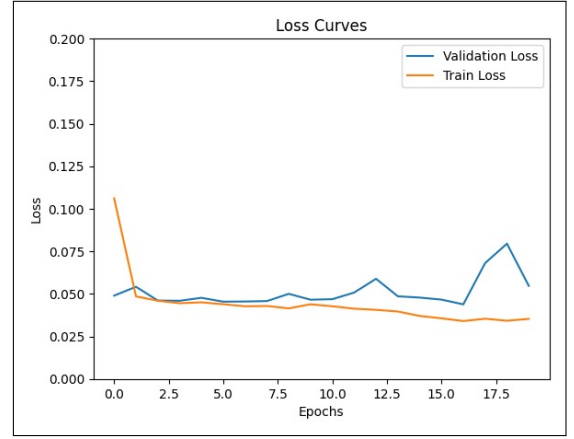


Fig. 7. Training and validation loss curves over 20 epochs, demonstrating rapid convergence with minimal overfitting.

93.10% validation accuracy demonstrates robust generalization to unseen power measurements. This performance enables practical automated key recovery: for an 8-bit exponent, the expected number of correctly classified bits is $8 \times 0.931 \approx 7.4$, meaning full key recovery succeeds with high probability after correcting only 0-1 bit errors through cryptographic validation.

The results confirm that deep learning can effectively automate side-channel cryptanalysis, transforming manual visual inspection, limited to toy examples, into a scalable attack capable of recovering production-length RSA keys from noisy analog measurements.

VI. CONCLUSION

This work demonstrates that co-resident malicious hardware on shared power infrastructure poses a realistic threat to cryptographic implementations on embedded systems. By emulating an attacker monitoring a shared power rail, we successfully extracted RSA exponents from an ATmega328P microcontroller through passive voltage measurements. KeySiphon, our automated deep learning pipeline, achieved 93.10% accuracy in classifying exponent bits from noisy power traces, demonstrating that side-channel attacks can scale beyond manual analysis to threaten production-length cryptographic keys.

These results expose a fundamental gap in IoT security: mathematical algorithm strength provides no protection against physical information leakage. The threat is particularly acute in multi-device deployments where a single compromised sensor or actuator can eavesdrop on neighboring devices through shared power distribution networks. Effective countermeasures must address the hardware-software interface through constant-time implementations, power consumption randomization, and secure power isolation in multi-tenant systems. As IoT deployments proliferate, understanding and mitigating shared infrastructure vulnerabilities becomes critical for maintaining cryptographic security guarantees in physically accessible environments.

REFERENCES

- [1] T. S. Messerges, E. A. Dabbish, and R. H. Sloan, "Power Analysis Attacks of Modular Exponentiation in Smartcards," in *Cryptographic Hardware and Embedded Systems - CHES 1999*, Lecture Notes in Computer Science, vol. 1717, pp. 144–157, Springer, 1999.
- [2] P. Kocher, J. Jaffe, and B. Jun, "Differential Power Analysis," in *Advances in Cryptology - CRYPTO '99*, Lecture Notes in Computer Science, vol. 1666, pp. 388–397, Springer, 1999.
- [3] M. M. Alani, "Side-Channel Attacks on IoT Devices: A Survey," *Electronics*, vol. 13, no. 21, pp. 4203, 2024.
- [4] T. S. Messerges, E. A. Dabbish, and R. H. Sloan, "Examining Smart-Card Security under the Threat of Power Analysis Attacks," *IEEE Transactions on Computers*, vol. 51, no. 5, pp. 541–552, May 2002.
- [5] S. Mangard, E. Oswald, and T. Popp, *Power Analysis Attacks: Revealing the Secrets of Smart Cards*, Springer, New York, 2007.
- [6] E. Brier, C. Clavier, and F. Olivier, "Correlation Power Analysis with a Leakage Model," in *Cryptographic Hardware and Embedded Systems - CHES 2004*, Lecture Notes in Computer Science, vol. 3156, pp. 16–29, Springer, 2004.
- [7] E. Oswald, "Enhancing Simple Power-Analysis Attacks on Elliptic Curve Cryptosystems," in *Cryptographic Hardware and Embedded Systems - CHES 2002*, Lecture Notes in Computer Science, vol. 2523, pp. 77–92, Springer, 2002.
- [8] P. C. Kocher, "Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems," in *Advances in Cryptology - CRYPTO '96*, Lecture Notes in Computer Science, vol. 1109, pp. 104–113, Springer, 1996.
- [9] S. Mangard, "A Simple Power-Analysis (SPA) Attack on Implementations of the AES Key Expansion," in *Information Security and Cryptology - ICISC 2002*, Lecture Notes in Computer Science, vol. 2587, pp. 343–358, Springer, 2003.
- [10] F.-X. Standaert, L. Oldenzeel, D. Samyde, and J.-J. Quisquater, "Power Analysis of FPGAs: How Practical Is the Attack?" in *Field Programmable Logic and Application - FPL 2003*, Lecture Notes in Computer Science, vol. 2778, pp. 701–711, Springer, 2003.
- [11] D. McCann, E. Oswald, and C. Whitnall, "Towards Practical Tools for Side Channel Aware Software Engineering: 'Grey Box' Modelling for Instruction Leakages," in *26th USENIX Security Symposium*, pp. 199–216, USENIX Association, 2017.
- [12] E. Cagli, C. Dumas, and E. Prouff, "Convolutional Neural Networks with Data Augmentation Against Jitter-Based Countermeasures," in *Cryptographic Hardware and Embedded Systems - CHES 2017*, Lecture Notes in Computer Science, vol. 10529, pp. 45–68, Springer, 2017.
- [13] C. Archambeau, E. Peeters, F.-X. Standaert, and J.-J. Quisquater, "Template Attacks in Principal Subspaces," in *Cryptographic Hardware and Embedded Systems - CHES 2006*, Lecture Notes in Computer Science, vol. 4249, pp. 1–14, Springer, 2006.
- [14] B. Timon, "Non-Profiled Deep Learning-Based Side-Channel Attacks with Sensitivity Analysis," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2019, no. 2, pp. 107–131, 2019.
- [15] Y. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Cross-VM Side Channels and Their Use to Extract Private Keys," in *Proceedings of the 2012 ACM Conference on Computer and Communications Security - CCS '12*, pp. 305–316, ACM, 2012.
- [16] Y. Yarom and K. Falkner, "FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack," in *23rd USENIX Security Symposium*, pp. 719–732, USENIX Association, 2014.